Leibniz FH

Wirtschaftsinformatik (B. Sc.)
Modul 20.1 (Vertiefung Entwicklung)
Sommersemester 2026

Anwendung erweiterter Softwareentwicklung

4. OOP (Objekt-orientierte Programmierung)

Dr. Peter Lohmann

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Leibniz FH

4.1. Konzepte

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Grundidee der Objektorientierung

- ✓ Objekte als gekapselte Einheiten
  - Zustand über (private) Attribute
  - Verhalten über Methoden
    - „Aufforderung ans Objekt“
- ✓ Motivation
  - mehrere gleichartige Dinge modellieren
  - Daten und Operationen zusammenhalten
    - Durchaus kontrovers: Kann Abhängigkeiten verschleiern

Klassen und Instanzen

- ✓ Klasse
  - Bauplan für Objekte
  - gemeinsame Struktur und gemeinsames Verhalten
- ✓ Instanz
  - konkretes Objekt zur Laufzeit
  - eigener Zustand pro Objekt
- ✓ JavaScript: Prototypen statt Klassen

Attribute, Konstruktoren und self

- ✓ Instanzattribute vs Klassenattribute
  - Daten pro Objekt vs globale Eigenschaften
- ✓ Instanzmethoden vs Klassenmethoden / statische Methoden
  - Verhalten pro Objekt vs gekapselte Funktionen
  - Python: Methode = Attribut, das Funktion ist
- ✓ `self`
  - Referenz auf das aktuelle Objekt
  - Dieser Variablenname ist nur Konvention!
- ✓ `__init__(self, ...)`
  - Initialisierung neuer Objekte
  - Läuft im Kontext des schon erzeugten Objekts
  - Magic (aka *dunder*) method

Referenzen und Kopien

- ✓ Sharing und Aliasing
  - Variablen speichern (nur) Referenzen auf Objekte!
  - mehrere Namen für dasselbe Objekt
  - automatische Destruktion, wenn keine Referenz mehr übrig
- ✓ Kopieren
  - shallow copy, deep copy

Leibniz FH

4.2. Polymorphie

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Subtyping

- ✓ Subtypen überall nutzbar, wo Supertyp erwartet wird
  - Python: Keine Garantie! Aber wichtiges Design-Prinzip
- ✓ gleiche Operation, verschiedene konkrete Objekte
- ✓ annotierter Typ
  - statische Beschreibung
  - Python: Annotation ohnehin bloß „netter Hinweis“
- ✓ Laufzeitobjekt
  - tatsächliches Verhalten

Dynamisches Binden

- ✓ Methodenwahl zur Laufzeit
- ✓ Auflösung über Objekt, Klasse und Basisklassen
- ✓ Nutzen
  - Erweiterbarkeit
  - Wiederverwendung
  - weniger Typfallunterscheidungen

Vererbung und super

- ✓ Subklassen übernehmen Attribute und Methoden
- ✓ Spezialisierung
  - Extend oder Override
- ✓ `super()`
  - vererbte Initialisierung
  - Zugriff auf Oberklassenverhalten
- ✓ Python
  - auch multiple Vererbung
  - MRO bestimmt Suchreihenfolge

Interfaces

- ✓ Python: *Abstract Base Class (ABC)*
- ✓ unvollständige Oberklassen
- ✓ abstrakte Methoden ohne Implementierung
- ✓ nicht direkt instanziierbar
- ✓ Nutzen
  - gemeinsame Verträge
  - klarere Entwürfe
  - Dependency Inversion

Overriding und Overloading

- ✓ Overriding
  - neue Implementierung geerbter Methoden
- ✓ Overloading
  - kein klassisches signature-basiertes Modell
  - meist Default-Werte oder flexible Parameter
- ✓ Shadowing
  - gleichnamige Namen überlagern vorhandene Definitionen